

实验四：内核线程管理

1 实验目的

本实验旨在深入理解并实现操作系统中内核线程的管理机制，包括进程控制块（PCB）的分配与初始化、内核线程的创建与资源分配、以及进程切换的实现。通过本实验，我们将掌握以下核心知识点：

- 1. **进程控制块（PCB）**：理解 `proc_struct` 结构体中各个字段的含义及其在进程生命周期中的作用。
- 2. **内核线程创建**：掌握 `do_fork` 函数的实现流程，理解如何通过复制父进程状态来创建子进程。
- 3. **进程切换**：深入理解 `switch_to` 函数的汇编实现，掌握上下文切换的底层机制。
- 4. **中断与同步**：理解在进程管理中如何使用关中断来保护临界区，防止竞态条件。

```
forkrets:
    move sp, a0      # 把sp设为传入的trapframe地址
    j __trapret      # 跳转到中断返回代码

__trapret:
    RESTORE_ALL      # 从trapframe恢复所有寄存器
    sret             # 返回到 tf->epc 所指地址
```

#新线程首次执行的完整流程

initproc 首次被调度执行的完整流程

【阶段1：调度决策】

```
idleproc 运行 cpu_idle()
|
V
schedule()      // 选择下一个进程
|
V
proc_run(initproc) // 切换到initproc
|
├─ current = initproc
├─ lsatp(initproc->pgdir)    // 切换页表
|
V
switch_to(&idle->context, &init->context)
```

【阶段2：上下文切换】

```
switch_to:
|
├─ STORE ra, sp, s0-s11 到 idle->context  // 保存idle的寄存器
|
├─ LOAD ra, sp, s0-s11 从 init->context    // 恢复init的寄存器
```

```

|
|   |
|   |— ra ← forkret (copy_thread设置的)
|   |— sp ← init->tf (trapframe位置)
|   |— s0 ← init_main, s1 ← "Hello world!!"
|
|— ret    // 跳转到 ra, 即 forkret

```

【阶段3: forkret → forkrets】

```

forkret():
|
|— forkrets(current->tf)    // 调用汇编函数
|
|   V
|   forkrets:
|       move sp, a0          // sp = trapframe地址
|       j __trapret

```

【阶段4: 中断返回】

```

__trapret:
|
|— RESTORE_ALL              // 从trapframe恢复寄存器
|
|   |
|   |— s0 ← init_main (函数指针)
|   |— s1 ← "Hello world!!" (参数)
|   |— sstatus ← SPP=1, SPIE=1 (S模式, 开中断)
|   |— sepc ← kernel_thread_entry
|
|— sret                      // 跳转到 sepc

```

【阶段5: 内核线程执行】

```

kernel_thread_entry:
|
|— move a0, s1              // a0 = "Hello world!!"
|
|— jalr s0                  // 调用 init_main(a0)
|
|   V
|   init_main("Hello world!!"):
|       cprintf("this initproc, pid = 1...")
|       cprintf("To U: Hello world!!")
|       return 0
|
|— jal do_exit              // 线程结束

```

【时间线总结】




```

proc->need_resched = 0;           // 不需要立即调度
proc->parent = NULL;             // 父进程：空
proc->mm = NULL;                 // 内存管理结构：空（内核线程不需要）
memset(&(proc->context), 0, sizeof(struct context)); // 上下文清零
proc->tf = NULL;                 // 中断帧指针：空
proc->cr3 = boot_cr3;           // 页目录基址：使用内核页目录
proc->flags = 0;                 // 标志位：0
memset(proc->name, 0, PROC_NAME_LEN); // 进程名清零
}

```

4 proc_struct 中 context 和 tf 的含义与作用

4.1 struct context context - 进程切换上下文

你的理解非常准确，已经抓住了这两个结构体的核心本质。不过为了让答案更加清晰、专业，并且更贴合 Lab4 的代码实现细节，我对你的回答进行了优化和补充。

以下是基于 Lab4 代码对 struct context context 和 struct trapframe *tf 的详细解释：

4.2 struct context context

含义：

context 是进程在内核态进行上下文切换时保存的寄存器状态。它只包含 RISC-V 调用约定中规定的**被调用者保存（Callee-saved）寄存器**。在 Lab4 的 kern/process/proc.h 和 kern/process/switch.S 中可以看到，它保存了以下 14 个寄存器：

- ra (Return Address): 返回地址，决定了切换回来后从哪里继续执行。
- sp (Stack Pointer): 栈指针，指向当前进程的内核栈。
- s0 ~ s11: 12 个被调用者保存的通用寄存器。

作用：

- 实现进程切换：**在 proc_run 函数中，通过调用 switch_to(&(prev->context), &(next->context))，内核会把当前 CPU 的寄存器状态保存到 prev->context 中，并将 next->context 中的值恢复到 CPU 寄存器中。
- 暂停与恢复执行流：**它记录了进程在内核态“暂停”时的瞬间状态。当进程再次被调度时，switch.S 会恢复这些寄存器，使得进程能够从上次暂停的地方（通常是 schedule() 函数调用之后）继续运行，就像从未被打断过一样。
- 新进程的入口引导：**对于新创建的内核线程，copy_thread 函数会将其 context.ra 设置为 forkret 函数的地址，context.sp 设置为指向内核栈顶的 tf。这样，当新进程第一次被 switch_to 选中时，它会“返回”到 forkret 函数，从而开始执行。

4.3 struct trapframe *tf

含义：

tf 是一个指针，指向进程**内核栈顶**的一个 struct trapframe 结构体。该结构体保存了进程在发生**中断、异常或系统调用那一瞬间的完整硬件现场**。它包含了 RISC-V 架构下的所有重要状态信息：

- **32 个通用寄存器 (gpr)**：包括 ra, sp, gp, tp, t0-t6, s0-s11, a0-a7。
- **特权级 CSR 寄存器**：sstatus (状态寄存器), sepc (异常程序计数器), sbadaddr (出错地址), scause (异常原因)。

作用：

1. **保存中断现场**：当进程在运行过程中发生中断（如时钟中断）或异常时，硬件和 trapentry.S 会将当前 CPU 的所有寄存器值压入内核栈，形成一个 trapframe。proc->tf 记录了这个结构体在栈上的位置，以便内核在处理完中断后能通过它恢复现场，回到中断前的状态。
2. **构造新进程的初始状态**：在 do_fork -> copy_thread 过程中，内核并不通过真正的中断进入，而是**人工伪造**了一个 trapframe 放在新进程的内核栈顶。
 - tf->gpr.a0 = 0：设置子进程 fork 的返回值为 0。
 - tf->gpr.sp：设置栈指针。
 - tf->epc：设置新进程真正开始执行的第一条指令地址（即 kernel_thread_entry）。
 - tf->status：设置特权级和中断使能状态。
3. **作为用户态与内核态的接口**：虽然 Lab4 只有内核线程，但在后续支持用户进程时，tf 是用户态进入内核态时保存用户寄存器的地方，也是内核态返回用户态时恢复用户寄存器的来源。

5 Exercise2: do_fork 函数的完整讲解 (Markdown)

本实验中的 do_fork() 是操作系统创建新进程的核心函数，其逻辑类似于 Linux 中的 do_fork()：通过复制或共享父进程资源、创建 PCB、设置上下文、加入调度系统，从而生成一个新的子进程。

本文分为两部分：

1. do_fork 主流程的完整讲解
2. 针对实验过程中的疑问逐条解析（来自同学提问）

5.1 do_fork 主流程讲解

5.1.1 函数目的

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
```

本函数负责：

- 创建进程控制块 (PCB)
- 为子进程分配内核栈

- 根据 clone_flags 决定共享/复制地址空间
- 设置子进程 trapframe（寄存器快照）
- 设置子进程 context（内核调度入口点）
- 将进程加入系统的进程链表/哈希表
- 把子进程设为 RUNNABLE（可调度运行）
- 返回子进程 PID

5.2 do_fork 流程逐步解析

5.2.1 步骤 0：检查系统是否还能创建进程

```
if (nr_process >= MAX_PROCESS)
    return -E_NO_FREE_PROC;
```

超过最大限制则失败。

5.2.2 步骤 1：分配进程控制块 PCB

```
proc = alloc_proc();
```

alloc_proc() 会分配一个空的 PCB，并初始化：

- state = UNINIT
- parent = current
- kstack = 0
- tf = NULL
- context 清空
- mm = NULL（后续 Lab 才会实现用户进程）

5.2.3 步骤 2：为新进程分配内核栈（kernel stack）

```
setup_kstack(proc);
```

关键点：每个进程都有自己独立的一块内核栈。

内核栈的用途：

- 该进程陷入内核态（中断/系统调用）时使用
- 内核函数调用使用的栈空间
- trapframe 就放在栈顶位置

其本质是在物理内存中 alloc_pages() 出一块，然后映射到内核虚拟地址空间。

5.2.4 步骤 3：复制或共享内存管理结构 mm

```
copy_mm(clone_flags, proc);
```

当前 Exercise2 中 mm==NULL，表示只有内核线程，因此 copy_mm 什么都不做：

```
assert(current->mm == NULL);
```

将来实现用户态 fork 时，该函数负责：

- CLONE_VM：父子共享地址空间（线程）
- 默认：复制地址空间（进程）

5.2.5 步骤 4：设置子进程的 trapframe 与 context

```
copy_thread(proc, stack, tf);
```

5.2.5.1 trapframe

```
proc->tf = (trapframe *) (proc->kstack + KSTACKSIZE - sizeof(struct trapframe));  
*(proc->tf) = *tf;
```

作用：

- trapframe 被放在 **内核栈顶**（从高地址往下）
- 复制父进程的寄存器快照
- a0 = 0 —— 子进程的 fork() 返回值为 0

5.2.5.2 设置子进程启动时的 sp

```
proc->tf->gpr.sp = (stack == 0) ? (uintptr_t)proc->tf : stack;
```

含义：

- stack==0 → 创建内核线程：使用 trapframe 作为启动栈
- stack!=0 → 未来用于用户线程：使用用户指定的用户态栈

5.2.5.3 设置 context

```
proc->context.ra = (uintptr_t)forkret;  
proc->context.sp = (uintptr_t)(proc->tf);
```

含义：

- 第一次调度到该进程时，会跳转到 forkret
- context.sp 指向 trapframe，最终会恢复寄存器 → 进入用户态

5.2.6 步骤 5：把进程加入系统结构（需要关闭中断）

```
local_intr_save(intr_flag);
{
    proc->pid = get_pid();
    hash_proc(proc);
    list_add(&proc_list, &(proc->list_link));
    nr_process++;
}
local_intr_restore(intr_flag);
```

为什么要关中断？

- PID 分配器是共享的
- 进程链表是共享的
- 进程哈希表是共享的
→ 如果中断在修改途中抢占，会破坏结构，必须保护临界区。

`local_intr_save` 会保存原中断状态并关闭中断；

`local_intr_restore` 会恢复原来的中断状态。

5.2.7 步骤 6：让进程进入 RUNNABLE 状态

```
wakeup_proc(proc);
```

这意味着：

- 调度器可以选择该进程执行
- 进程运行时从 `context.ra`（forkret）开始

5.2.8 步骤 7：返回子进程 PID

父进程得到子 PID；

子进程通过 `trapframe` 修正返回值为 0。

5.3 do_fork 调用链结构图

```
do_fork
|
|— alloc_proc
|
|— setup_kstack
|
|— copy_mm
|
|— copy_thread
|   |— 设置 tf（内核栈顶）
|   |— gpr.sp = ...（子进程启动栈）
|   |— 设置 context
|
```



```
└─ local_intr_save()
|   └─ get_pid
|   └─ hash_proc
|   └─ list_add
|   └─ nr_process++
|   local_intr_restore()
|
└─ wakeup_proc
```

5.4 回答一些问题

5.4.1 copy_thread 中这行代码什么意思？

```
proc->tf = (proc->kstack + KSTACKSIZE - sizeof(struct trapframe));
```

不是申请内存！不是创建一大块区域！

它只是让 tf 指向：

```
内核栈的最高地址 - trapframe 大小
```

也即 **trapframe** 被放在内核栈顶附近。

- `proc->kstack` = 栈底
- `proc->kstack + KSTACKSIZE` = 栈顶
- `trapframe` 需要栈顶留出一块空间，因此减去 `sizeof(struct trapframe)`

5.4.2 为什么 trapframe 要放在内核栈顶？

因为：

- 每个进程进入内核态必须保存寄存器现场（ra/sp/a0/pc 等）
- `trapframe` 必须是“属于这个进程的、独立的结构”
- 最自然的位置就是 **这个进程自己的内核栈的最顶部**

恢复进程时：

- `context.sp` 指向 `trapframe`
- `ret` 到 `forkret`
- `forkret` 最终恢复寄存器 → 回到用户态

5.4.3 esp 参数有什么用途？

代码：

```
proc->tf->gpr.sp = (esp == 0) ? (uintptr_t)proc->tf : esp;
```

实验阶段（只有内核线程）：

- esp==0 → 使用 trapframe 所在位置作为启动栈

未来（有用户进程）：

- clone(thread) 或 fork 时可以传入用户栈地址
→ 子进程/线程使用新的用户栈

5.4.4 gpr 是否是“真正的寄存器”？

不是。

- gpr 是 trapframe 结构体中的字段
- 是 **CPU 寄存器的一份备份（snapshot）**
- 存放在内核栈中，属于每个进程独立拥有

真正的寄存器在 CPU 内核中；

当调度器切换进程时，会把 gpr 恢复进真正寄存器。

5.4.5 内核栈到底是什么？

每个进程都有 **自己的内核栈**：

- 进入内核（中断/系统调用）时使用
- 内核函数调用的栈帧存放在这里
- trapframe 也放在内核栈顶

关键点：

- **每个进程都有一份独立的物理内核栈**
- 它们不会共享
- 只有内核代码在访问它们

5.4.6 local_intr_save(flag) 怎么给 flag 赋值？

因为它是宏：

```
##define local_intr_save(x) x = __intr_save()
```

展开后就是：

```
intr_flag = __intr_save();
```

__intr_save() 会：

- 检查当前是否开启中断
- 如果开启 → 关闭它并 return 1
- 如果关闭 → return 0

所以：

- flag=1 → 原来中断开 → 临界区后要恢复开
- flag=0 → 原来中断关 → 临界区后保持关

5.5 总结

do_fork 的核心任务是：

- 创建 PCB
- 创建内核栈
- 复制寄存器快照 (trapframe)
- 设置 context (第一次运行的入口)
- 将新进程加入调度系统
- 设置为 RUNNABLE

并保证整个过程的正确性 (中断保护、错误回滚)。

实验中由于还没有 mm (用户地址空间)，所以只涉及 **内核线程** 的创建，但框架已经体现了真实 OS 的完整 fork/clone 机制。

6 练习3：编写proc_run函数

6.1 设计实现过程

proc_run 函数是进程调度的核心执行单元，它的任务是将 CPU 的控制权从当前进程 (current) 转移到指定的目标进程 (proc)。这个过程涉及到 CPU 状态的保存与恢复、地址空间的切换以及内核栈的切换。

6.1.1 详细步骤解析：

1. 检查是否需要切换：

- if (proc != current): 首先判断目标进程是否就是当前进程。如果是，则无需进行任何操作，直接返回。这是一种常见的优化。

2. 保护临界区 (关中断)：

- local_intr_save(intr_flag): 在进行上下文切换之前，必须关闭中断。
- **原因：**上下文切换是一个敏感的系统状态变更过程。如果在切换过程中 (例如，已经修改了 current 指针，但还没切换栈) 发生了中断，中断处理程序可能会依赖 current 指针访问数据，导致状态不一致甚至内核崩溃。此外，我们不希望切换过程中被嵌套的调度打断。

3. 更新当前进程指针：

- struct proc_struct *prev = current;: 保存当前进程的指针，稍后需要保存它的上下文。
- struct proc_struct *next = proc;: 目标进程。

- `current = proc;`: 将全局变量 `current` 更新为目标进程。从这一刻起，逻辑上 OS 认为当前运行的是 `proc`，尽管 CPU 寄存器还没切换。

4. 切换页表（地址空间）:

- `lsatp(proc->pgdir)`: 这是 RISC-V 特有的操作。`satp` 寄存器保存了根页表的物理地址。
- **为什么要切换?**: 虽然内核线程共享内核地址空间（即页表的高地址部分映射相同），但为了统一处理（未来会有用户进程，它们有独立的页表），以及确保 TLB（Translation Lookaside Buffer）的正确性，我们需要切换到新进程的页表。
- **TLB 刷新**: 写入 `satp` 寄存器通常需要配合 `sfence.vma` 指令来刷新 TLB，以防止旧进程的地址转换缓存影响新进程。`lsatp` 函数内部封装了写寄存器操作，硬件或后续指令通常会处理 TLB 刷新。

5. 上下文切换（Context Switch）:

- `switch_to(&(prev->context), &(next->context))`: 这是最神奇的一步。
- 该函数由汇编实现。它接受两个参数：旧进程的上下文指针和新进程的上下文指针。
- **保存**: 它将当前 CPU 的 `ra`, `sp`, `s0-s11` 寄存器保存到 `prev->context` 中。注意，这里的 `ra` 是 `proc_run` 函数中调用 `switch_to` 后的下一条指令地址。
- **恢复**: 它从 `next->context` 中加载寄存器值到 CPU。
- **跳转**: 最后执行 `ret` 指令。由于 `ra` 寄存器已经被恢复为新进程上次保存的值（或者是 `forkret`），CPU 将跳转到新进程的代码处继续执行。
- **视角的转换**: 对于 `prev` 进程，它停在了 `switch_to` 调用处；对于 `next` 进程，它仿佛刚从 `switch_to` 返回。

6. 恢复中断:

- `local_intr_restore(intr_flag)`: 当进程切换回来后（即 `prev` 再次变成 `current` 时），代码会从 `switch_to` 后面继续执行，此时恢复中断状态。

6.2 5.2 关键代码实现

```
void proc_run(struct proc_struct *proc)
{
    if (proc != current)
    {
        // LAB4:EXERCISE3 2312325
        bool intr_flag;
        struct proc_struct *prev = current, *next = proc;

        // 1. 关中断，保护进程切换过程
        local_intr_save(intr_flag);
        {
            // 2. 更新 current 指针
            current = proc;

            // 3. 切换页表：设置新进程的页目录表地址到satp寄存器
            // 这会刷新 TLB，确保虚拟地址到物理地址的映射正确
        }
    }
}
```

```

lsatp(proc->pgdir);

// 4. 切换上下文：从当前进程切换到新进程
//    这里会保存 prev 的寄存器，恢复 next 的寄存器
//    执行完这行代码后，CPU 就开始执行 next 进程的代码了
switch_to(&(prev->context), &(next->context));
}
// 5. 恢复中断
//    注意：这行代码是在进程再次被调度回来时才会执行
local_intr_restore(intr_flag);
}
}

```

6.3 问题回答

在本实验的执行过程中，创建且运行了几个内核线程？

答：创建并运行了2个内核线程。

详细分析：

1. idleproc（空闲线程，PID=0）：

- **创建：**在 `proc_init` 函数开始时，通过 `alloc_proc` 直接分配内存并手动初始化。
- **特点：**它的 `pid` 被强制设为 0，状态设为 `PROC_RUNNABLE`，内核栈指向 `bootstack`。
- **作用：**它是系统的第 0 个进程，也是当系统中没有其他可运行进程时 CPU 的“归宿”。
- **运行：**`kern_init` 最后调用 `cpu_idle()`，实际上就是 `idleproc` 在运行。它在一个死循环中不断查询 `need_resched` 标志，一旦有其他进程准备好（如 `initproc`），就调用 `schedule()` 让出 CPU。

2. initproc（初始化线程，PID=1）：

- **创建：**在 `proc_init` 中，通过调用 `kernel_thread(init_main, ...)` 创建。
- **底层机制：**`kernel_thread` 内部调用 `do_fork`，`do_fork` 会分配新的 PCB，复制父进程（此时是 `idleproc`）的信息，并设置新的内核栈和上下文。
- **作用：**它是系统的第一个实际工作线程。在本实验中，它的任务很简单，就是打印 "Hello world!!"。在后续实验中，它将负责创建用户进程。
- **运行：**当 `idleproc` 调用 `schedule()` 时，调度器发现 `initproc` 处于 `PROC_RUNNABLE` 状态，于是调用 `proc_run` 切换到 `initproc`。

执行流程复盘：

1. OS 启动，`kern_init` 运行（此时使用的是启动栈）。
2. `proc_init` 创建 `idleproc`，并将 `current` 指向它。
3. `proc_init` 调用 `kernel_thread` 创建 `initproc`。`initproc` 被加入运行队列，状态为 `RUNNABLE`。
4. `kern_init` 结束，进入 `cpu_idle`（即 `idleproc` 的主循环）。

- 最终的完整的运行结果如下:

OpenSBI v0.4 (Jul 2 2019 11:53:53)

```
Platform Name      : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 112 KB
Runtime SBI Version  : 0.1
```

```
Special kernel symbols:
    entry   0xc020004a (virtual)
    etext   0xc0203ec6 (virtual)
    edata    0xc0209030 (virtual)

Kernel executable memory footprint: 54KB
memory management: default_pmm_manager

physcial memory map:
physcial memory map:
physcial memory map:
    memory: 0x08000000, [0x80000000, 0x87ffffff].
```

```
vapaofset is 18446744070488326144
s
s
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
kernel panic at kern/process/proc.c:412:
    process exit!!.

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
```

```
root@song:/mnt/d/gds/Documents/Operating_system/lab4# make grade
gmake[1]: Warning: File 'obj/libs/hash.d' has modification time 0.046 s in the future
riscv64-unknown-elf-ld: removing unused section '.rodata.__warn.str1.8' in file
'obj/kern/debug/panic.o'
riscv64-unknown-elf-ld: removing unused section '.text.__warn' in file 'obj/kern/debug/panic.o'
riscv64-unknown-elf-ld: removing unused section '.text.is_kernel_panic' in file
'obj/kern/debug/panic.o'
riscv64-unknown-elf-ld: removing unused section '.text.kbd_intr' in file 'obj/kern/driver/console.o'
riscv64-unknown-elf-ld: removing unused section '.text.serial_intr' in file
'obj/kern/driver/console.o'
riscv64-unknown-elf-ld: removing unused section '.text.pic_enable' in file
'obj/kern/driver/picirq.o'
riscv64-unknown-elf-ld: removing unused section '.text.slob_init' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.slob_allocated' in file
'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.kallocated' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.ksize' in file 'obj/kern/mm/kmalloc.o'
riscv64-unknown-elf-ld: removing unused section '.text.tlb_invalidate' in file 'obj/kern/mm/pmm.o'
riscv64-unknown-elf-ld: removing unused section '.rodata.print_vma.str1.8' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.print_vma' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.rodata.print_mm.str1.8' in file
'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.print_mm' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.mm_create' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.vma_create' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.mm_destroy' in file 'obj/kern/mm/vmm.o'
riscv64-unknown-elf-ld: removing unused section '.text.set_proc_name' in file
'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.get_proc_name' in file
'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.find_proc' in file 'obj/kern/process/proc.o'
riscv64-unknown-elf-ld: removing unused section '.text.sprintputch' in file 'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.snprintf' in file 'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.vsnprintf' in file 'obj/libs/printfmt.o'
riscv64-unknown-elf-ld: removing unused section '.text.strncpy' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strfind' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.strtol' in file 'obj/libs/string.o'
riscv64-unknown-elf-ld: removing unused section '.text.memmove' in file 'obj/libs/string.o'
```

```
gmake[1]: warning: Clock skew detected. Your build may be incomplete.
gmake[1]: Entering directory '/mnt/d/gds/Documents/Operating_system/lab4' + cc kern/init/entry.S +
cc kern/init/init.c + cc kern/libs/readline.c + cc kern/libs/stdio.c + cc kern/debug/kdebug.c + cc
kern/debug/kmonitor.c + cc kern/debug/panic.c + cc kern/driver/clock.c + cc kern/driver/console.c +
cc kern/driver/dtb.c + cc kern/driver/intr.c + cc kern/driver/picirq.c + cc kern/trap/trap.c + cc
kern/trap/trapentry.S + cc kern/mm/default_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc
kern/mm/vmm.c + cc kern/process/entry.S + cc kern/process/proc.c + cc kern/process/switch.S + cc
kern/schedule/sched.c + cc libs/hash.c + cc libs/printfmt.c + cc libs/string.c + ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img gmake[1]: Leaving
directory '/mnt/d/gds/Documents/Operating_system/lab4'
-check alloc proc: OK
-check initproc: OK
Total Score: 30/30
```

7 扩展练习 Challenge

7.1 6.1 说明语句

local_intr_save(intr_flag);...local_intr_restore(intr_flag); 是如何实现开关中断的？

这是一个非常经典且巧妙的设计，用于在内核中实现可嵌套的临界区保护。

7.1.1 核心问题：为什么不能直接用 cli() 和 sti()？

假设我们有两个函数 A 和 B，A 调用 B：

```
void A() {
    cli(); // 关中断
    B();
    sti(); // 开中断
}

void B() {
    cli(); // 关中断
    // ... 临界区操作 ...
    sti(); // 开中断
}
```

当 A 调用 B 时，B 执行完 sti() 后，中断就被打开了。但是 A 还在临界区内！A 希望中断一直关闭直到它自己执行 sti()。直接使用开关中断指令会导致**过早开启中断**，破坏临界区的原子性。

7.1.2 解决方案：保存中断状态

local_intr_save(x) 和 local_intr_restore(x) 的核心思想是：**在关中断之前，先保存当前的中断状态（是开还是关）；在恢复时，不是盲目打开中断，而是恢复到之前保存的状态。**

7.1.3 实现原理与代码分析

在 RISC-V 架构下，这两个宏的展开逻辑如下：

local_intr_save(x):

```
#define local_intr_save(x) \
do { \
    x = __intr_save(); \
} while (0)

static inline bool __intr_save(void) {
    if (read_csr(sstatus) & SSTATUS_SIE) { // 1. 读取 sstatus 寄存器，检查 SIE 位（中断使能位）
        intr_disable(); // 2. 如果 SIE 为 1（开中断），则执行关中断指令（csrr sstatus, SSTATUS_SIE）
        return 1;        // 3. 返回 1，表示“之前是开中断的”
    }
    return 0;            // 4. 如果 SIE 为 0（本来就是关的），则不操作，返回 0
}
```

- **关键点：**变量 x（即 intr_flag）保存了**进入临界区之前**的中断状态。

local_intr_restore(x):

```
#define local_intr_restore(x) \
__intr_restore(x)

static inline void __intr_restore(bool flag) {
    if (flag) { // 1. 检查之前保存的状态
        intr_enable(); // 2. 如果之前是开中断的（flag=1），那么现在重新开启中断（csrr sstatus, SSTATUS_SIE）
    }
    // 3. 如果之前是关中断的（flag=0），那么什么都不做，保持关中断状态。
}
```

7.1.4 嵌套场景推演

回到刚才 A 调用 B 的例子：

1. **进入 A:** local_intr_save(flag_A)。假设此时中断是开的。flag_A 被设为 1，中断被关闭。
2. **A 调用 B。**
3. **进入 B:** local_intr_save(flag_B)。此时中断已经是关的（A 关的）。read_csr 发现 SIE 是 0。flag_B 被设为 0。中断保持关闭。
4. **B 执行...**
5. **退出 B:** local_intr_restore(flag_B)。flag_B 是 0。函数检查 if(0)，不执行 intr_enable()。**中断保持关闭！**这正是我们想要的。
6. **回到 A...**
7. **退出 A:** local_intr_restore(flag_A)。flag_A 是 1。函数执行 intr_enable()。中断被正确重新开启。

通过这种方式，`intr_flag` 实际上在栈上形成了一个“中断状态栈”，完美解决了嵌套调用的问题。

7.2 深入理解不同分页模式的工作原理

`get_pte()`函数中有两段形式类似的代码，结合sv32，sv39，sv48的异同，解释这两段代码为什么如此相像。

7.2.1 多级页表的通用逻辑

RISC-V 定义了 Sv32（2级页表）、Sv39（3级页表）和 Sv48（4级页表）等多种分页模式。尽管级数不同，但它们的设计遵循完全相同的**递归查找逻辑**：

- **结构一致性**：每一级页表（Page Directory/Table）都占用一个物理页（4KB）。
- **条目一致性**：页表项（PTE）的格式高度统一，都包含物理页号（PPN）和标志位（V, R, W, X, U 等）。
- **查找算法一致性**：
 1. 从根页表（由 `satp` 指定）开始。
 2. 利用虚拟地址的高位段（VPN[i]）作为索引，找到对应的 PTE。
 3. 检查 PTE 的有效位（V）。
 4. 如果 PTE 指向下一级页表（R=W=X=0），则取出 PPN 作为下一级页表的基址，重复步骤 2。
 5. 如果 PTE 是叶子节点（R/W/X 不全为0），则完成翻译。

7.2.2 代码相似性分析

在 `get_pte` 函数中，我们需要模拟硬件的页表遍历过程来查找或创建页表项。

对于 Sv39（3级页表）：

- **第一段代码**：处理一级页表（PDT1）。使用 `PDX1(1a)` 获取索引，查找 PTE。如果不存在且需要创建，则分配物理页，设置 PTE 指向它。
- **第二段代码**：处理二级页表（PDT0）。使用 `PDX0(1a)` 获取索引，查找 PTE。逻辑与第一段完全相同：查表 -> 判空 -> 分配 -> 链接。

这两段代码之所以如此相像，是因为**它们在做完全相同的事情，只是针对的页表层级不同**。Sv39 相比 Sv32 只是多了一层中间页表，因此代码中就多了一段类似的逻辑。如果是 Sv48，就会有三段类似的代码。

目前`get_pte()`函数将页表项的查找和页表项的分配合并在一个函数里，你认为这种写法好吗？有没有必要把两个功能拆开？

观点：这种写法是合理的，符合内核开发的惯例，但在特定场景下拆开也有其优势。

7.2.3 合并的优点 (Current Approach):

1. **原子性与便利性**: 在内核内存管理中, 最常见的操作是“给我这个虚拟地址对应的 PTE, 如果不存在就帮我造一个”。`get_pte(..., create=1)` 完美封装了这个语义。调用者 (如 `page_insert`) 不需要关心中间页表是否存在, 也不需要编写复杂的 `if-else` 逻辑来处理缺页。
2. **性能优化**: 如果拆成 `lookup` 和 `alloc` 两个函数。当 `lookup` 失败时, 我们调用 `alloc`。`alloc` 必须重新遍历页表才能找到插入点 (因为 `lookup` 通常只返回结果, 不返回中间状态)。这会导致重复的页表遍历, 浪费 CPU 周期。合并写法可以在一次遍历中完成查找和分配。
3. **代码紧凑**: 减少了函数定义和调用的开销。

7.2.4 拆开的优点 (Alternative Approach):

1. **单一职责原则 (SRP)**: 查找是查询操作 (无副作用), 分配是修改操作 (有副作用)。拆开后函数功能更纯粹, 易于单元测试和理解。
2. **安全性**: 某些场景下 (如调试器查看内存、性能统计), 我们绝对不希望因为查询而意外分配内存。虽然可以通过 `create=0` 参数控制, 但拆开后的接口 (如 `get_pte_readonly`) 在语义上更安全, 防止误用。

7.2.5 结论

在操作系统内核这种对**性能**和**代码效率**要求极高的环境下, **合并写法是更优的选择**。它减少了冗余计算, 提供了强大的接口能力。只要参数命名清晰 (如 `bool create`), 其带来的便利性远大于违背单一职责原则带来的困扰。实际上, Linux 内核中的页表操作也广泛采用了类似的模式。

8 重要知识点总结

8.1 实验知识点与OS原理对照

实验知识点	OS原理知识点	含义、关系与差异
进程控制块 (PCB)	进程描述符/PCB	实验中为 <code>struct proc_struct</code> , 包含进程状态、PID、上下文等信息; 原理中为抽象概念, 用于描述和管理进程的所有信息。实验是原理的具体实现。
上下文切换 (context)	进程上下文切换	实验中通过 <code>context</code> 结构体保存 <code>ra</code> 、 <code>sp</code> 、 <code>s0-s11</code> 等寄存器, 并通过 <code>switch_to</code> 汇编实现切换; 原理中为保存/恢复CPU寄存器状态的抽象过程。实验展示了底层硬件级的实现细节。
内核线程创建	进程/线程创建机制	实验中通过 <code>do_fork</code> 实现, 包括分配PCB、内核栈、设置 <code>trapframe</code> 等; 原理中为 <code>fork</code> 系统调用的抽象描述。实验侧重于内核态实现, 原理包含用户态和内核态。
进程调度 (schedule)	进程调度算法	实验中使用简单的轮转调度, 遍历进程链表选择下一个 <code>RUNNABLE</code> 进程; 原理中为多种调度算法 (FCFS、SJF、优先级、多级反馈队列等)。实验是最简单的实现。
trapframe (中断帧)	中断/异常处理	实验中 <code>trapframe</code> 保存中断发生时的CPU状态, 用于中断返回和线程启动; 原理中为中断处理机制的一部分。实验展示了具体的数据结构和使用方式。
进程状态转换	进程状态模型	实验中定义了 <code>UNINIT</code> 、 <code>SLEEPING</code> 、 <code>RUNNABLE</code> 、 <code>ZOMBIE</code> 四种状态; 原理中通常包含新建、就绪、运行、阻塞、终止五种状态。实验简化了状态模型, 合并了部分状态。
唤醒机制 (wakeup_proc)	进程阻塞与唤醒	实验中 <code>wakeup_proc</code> 将进程状态设为 <code>RUNNABLE</code> , 使其可被调度; 原理中为将阻塞进程移入就绪队列的操作。实验是原理的直接实现。
内核栈 (kstack)	进程内核栈	实验中为每个进程分配独立的内核栈, 用于中断处理和内核态执行; 原理中为进程在内核态运行时使用的栈空间。实验展示了具体的分配和使用方式。

8.2 OS原理中重要但实验未涉及的知识点

1. 用户态与内核态切换:

- 实验中只实现了内核线程, 它们始终运行在 S-Mode (内核态)。
- 原理中, 真正的进程通常运行在 U-Mode, 通过系统调用 (`ecall`) 陷入内核。这涉及特权级切换、用户栈到内核栈的切换 (`sscratch`寄存器的使用) 等复杂机制。

2. 进程间通信 (IPC):

- 实验中未实现任何IPC机制（管道、消息队列、共享内存、信号量等）。
- 原理中，进程间通常是隔离的，必须通过 IPC 进行协作。

3. 进程同步与互斥：

- 实验中虽然有中断开关的原子操作，但未涉及信号量、互斥锁、条件变量等高级同步原语。
- 原理中，进程同步是解决竞态条件、死锁等问题的关键。

4. 虚拟内存的高级管理：

- 实验中虽然有页表切换，但未深入实现缺页异常（Page Fault）、页面置换算法（如 LRU）、写时复制（Copy-on-Write）等。
- 原理中，这些是虚拟内存系统的核心，允许系统运行比物理内存更大的程序。

5. 多核调度（SMP）：

- 实验基于单核 CPU。
- 原理中，多核调度涉及负载均衡、CPU 亲和性、跨核中断（IPI）等复杂问题。

9 实验总结

通过本次实验，我深入理解了操作系统内核最核心的机制之一——进程管理。

1. 从数据结构到动态流程：

- 以前对 PCB 的理解仅停留在书本上的字段定义。通过实现 `alloc_proc`，我明白了每个字段的具体用途，特别是 `context` 和 `trapframe` 的区别：一个用于线程切换（主动），一个用于中断现场（被动）。

2. 上下文切换的“魔法”：

- `switch_to` 函数的实现让我大开眼界。它利用栈和寄存器的配合，实现了执行流的“瞬移”。理解了“调用一次，返回两次”（分别在父子进程中）的底层原理。

3. 临界区保护的重要性：

- 在实现 `do_fork` 和 `proc_run` 时，必须时刻警惕中断的影响。`local_intr_save` 的嵌套设计让我领略了内核代码的精妙之处。

4. 调试的艰辛与收获：

- 实验过程中，曾遇到 `make qemu` 卡死的情况。通过分析发现是 `riscv.h` 中 `lsatp` 宏对 64 位分页模式支持不足（使用了 32 位的常量）。修复这个 Bug 的过程让我对 RISC-V 的特权级架构有了更深的认识。

本次实验不仅完成了代码填空，更是一次对操作系统底层逻辑的深度探索，为后续实现用户进程和文件系统打下了坚实的基础。